ACID Transactions

Contents

1. What a transaction is

A transaction is a group of statements the database treats as one unit of work. ACID is the set of four promises the database makes about that unit: **A**tomicity, **C**onsistency, **I**solation, **D**urability.

```
BEGIN;
  UPDATE accounts SET balance = balance - 100 WHERE id = 1;
  UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;
```

Either both updates take effect or neither does. There is no state where the money has left one account without arriving in the other.

Every statement runs inside a transaction, whether we open one or not. Without an explicit `BEGIN`, the engine wraps each statement in its own transaction and commits it right away. That mode is called autocommit, and it is the default in Postgres, MySQL and SQL Server.

`ROLLBACK` throws the work away instead of committing it. A crash in the middle of a transaction has the same effect: on restart the engine rolls back anything that never reached `COMMIT`.

2. Atomicity

A transaction is a single unit of work. Either every statement in it completes, or none of them do. A failure halfway through leaves no trace of the statements that already ran.

The engine does this by writing down how to undo each change before it makes the change. Postgres keeps the old row versions in the table itself; MySQL InnoDB writes them to a separate undo log. Either way, a rollback puts back the previous version of every row the transaction touched.

What atomicity covers:

- An error on the third of five statements. Nothing from the first two survives.
- A client that drops the connection mid-transaction. The engine rolls back on its behalf.
- A process crash or power cut. Recovery rolls back every transaction that had not committed.

A savepoint lets us undo part of a transaction instead of all of it. That helps when one optional step is allowed to fail:

```
BEGIN;
  INSERT INTO orders (user_id, total) VALUES (42, 250);
  SAVEPOINT after_order;
  INSERT INTO loyalty_points (user_id, points) VALUES (42, 25);
  ROLLBACK TO SAVEPOINT after_order;  -- drop the points, keep the order
COMMIT;
```

> **Worth knowing**
>
> Most engines commit on their own the moment we run DDL such as `CREATE TABLE` or `ALTER TABLE`, so that change cannot be rolled back. Postgres is the exception: its DDL sits inside the transaction, which is why migrations there can be wrapped in `BEGIN`. MySQL and Oracle commit as soon as the DDL runs, which quietly commits any pending work along with it.

3. Consistency

A transaction moves the database from one valid state to another valid state. "Valid" means every rule holds: primary keys, foreign keys, unique constraints, check constraints, and the rules our own application cares about.

```
ALTER TABLE accounts ADD CONSTRAINT balance_non_negative CHECK (balance >= 0);
```

With that constraint in place, no transaction can commit a negative balance. The write is rejected and the transaction rolls back.

This is the letter interviewers ask the most follow-up questions about, because the database only owns half of it. The engine enforces the rules we wrote down. It cannot enforce the rules that live only in our heads or in our service code. "Every order has at least one line item" is a consistency rule, but unless we write it as a constraint or a trigger, nothing stops a transaction from breaking it.

A better way to say it: consistency is the result, and the other three letters are how we get there. Atomicity stops half-finished work from being saved, isolation stops parallel transactions from spoiling each other's work, durability stops committed work from disappearing. Consistency is what we get when those three hold and we wrote our constraints down properly.

4. Isolation

Transactions running at the same time should not see each other's unfinished work. The strongest version of the promise: running them side by side gives the same result as running them one after another, in some order.

Full isolation is expensive, so every engine offers weaker settings. They let more work run at once, and in exchange they let specific problems through. Sections 6 to 8 cover that trade, and it is where most interview follow-up questions lead.

5. Durability

Once `COMMIT` returns, the change survives a crash. Pulling the power out of the machine one millisecond later does not lose it.

Engines do this with a write-ahead log. Before touching the real data pages, the engine adds a note about the change to a log file, then calls `fsync` to push that note out of the operating system cache and onto the disk. Only then does `COMMIT` return. If the process dies before the data pages are written, recovery replays the log and rebuilds them.

Durability is only as strong as that `fsync`, and both major engines let us weaken it for speed:

Setting	Effect
`synchronous_commit = off` (Postgres)	`COMMIT` returns before the log is flushed. A crash can lose the last few hundred milliseconds of committed transactions.
`innodb_flush_log_at_trx_commit = 2` (MySQL)	Log is written to the OS cache on commit, flushed once per second. Survives a process crash, not a machine crash.

Once we add replicas, durability gains a second question: how many machines hold the change before commit returns? A commit that is safe on one machine and lost when another takes over was not durable in the way anyone cared about.

6. What goes wrong without isolation

Four classic problems, called anomalies, in the order they usually get asked about.

Dirty read. We read a row another transaction has written but not committed. If that transaction rolls back, we acted on a value that never existed.

```
T1: UPDATE accounts SET balance = 50 WHERE id = 1;   -- not committed
T2: SELECT balance FROM accounts WHERE id = 1;       -- reads 50
T1: ROLLBACK;                                        -- the 50 never existed
```

Non-repeatable read. We read the same row twice in one transaction and get different values, because another transaction committed a change in between.

```
T1: SELECT balance FROM accounts WHERE id = 1;       -- 100
T2: UPDATE accounts SET balance = 50 WHERE id = 1; COMMIT;
T1: SELECT balance FROM accounts WHERE id = 1;       -- 50, in the same transaction
```

Phantom read. We run the same query twice and get a different *set* of rows, because another transaction added or removed rows that match our filter.

```
T1: SELECT count(*) FROM orders WHERE total > 100;   -- 12
T2: INSERT INTO orders (total) VALUES (500); COMMIT;
T1: SELECT count(*) FROM orders WHERE total > 100;   -- 13
```

Lost update. Two transactions read the same row, both work out a new value from what they read, and the second write overwrites the first.

```
T1: SELECT balance FROM accounts WHERE id = 1;       -- 100
T2: SELECT balance FROM accounts WHERE id = 1;       -- 100
T1: UPDATE accounts SET balance = 90 WHERE id = 1; COMMIT;
T2: UPDATE accounts SET balance = 80 WHERE id = 1; COMMIT;  -- T1's deduction is gone
```

Lost update causes the most trouble in real code, and the fix is not always a higher isolation level. Doing the math in the database (`SET balance = balance - 10`) instead of in our own code closes the gap between the read and the write.

7. The four isolation levels

The SQL standard defines four levels. What separates them is which problems each one still lets through.

Level	Dirty read	Non-repeatable read	Phantom read
Read uncommitted	Possible	Possible	Possible
Read committed	Blocked	Possible	Possible
Repeatable read	Blocked	Blocked	Possible
Serializable	Blocked	Blocked	Blocked

Reading down the table, each level blocks one more problem and lets less work run at the same time. Setting the level is one statement:

```
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

The standard says what each level must *block*, not what it must *allow*. An engine is free to be stricter than its label, and several are.

8. What the engines actually do

The labels are a minimum, not a full description. Three differences are worth knowing before an interview.

Postgres never gives dirty reads. Asking for `READ UNCOMMITTED` is accepted, but the transaction quietly runs as `READ COMMITTED` anyway. Postgres has no way to show uncommitted rows, so the weakest level cannot be used.

Defaults differ. Postgres defaults to `READ COMMITTED`. MySQL InnoDB defaults to `REPEATABLE READ`. The same code behaves differently on the two engines with nothing in it changed, which is worth remembering before moving code between them.

Repeatable read is stronger than the label. Postgres builds it as snapshot isolation: the transaction sees one fixed view of the database as it was at the first read, so phantoms cannot appear either. MySQL InnoDB gets the same result for ordinary reads, and uses gap locks to block phantoms on locking reads such as `SELECT ... FOR UPDATE`.

Both engines rely on **MVCC**, multi-version concurrency control. Instead of locking readers out while a writer works, the engine keeps several versions of each row and shows each transaction the versions that were current when its snapshot began. Readers never block writers and writers never block readers, which is why these databases stay quick when reads and writes are mixed together.

> **Write skew, and why serializable still exists**
>
> Snapshot isolation blocks all three standard anomalies but still lets **write skew** through. Two transactions read an overlapping set of rows, each checks a rule that holds right now, and each writes a *different* row. Both commit, and together they break the rule that each one kept on its own.
>
> The classic case is an on-call schedule that needs at least one doctor on shift. Two doctors are on, and both want to go home. Each opens a transaction, counts two doctors on shift, decides that leaving is safe, and clears their own row. Both counts were correct when they ran. Nobody is left on call, and no single transaction did anything wrong. Postgres `SERIALIZABLE` catches this by tracking which live transactions read and wrote the same data, then cancelling one of them with SQLSTATE `40001`.

So in Postgres, serializable costs us in our own code rather than in lock waits. Transactions can fail at commit time with a serialization error, and every caller has to be ready to retry the whole transaction.

9. A transfer in SQL

The classic example, with the two things that actually make it safe.

```sql
BEGIN;

UPDATE accounts
SET balance = balance - 100
WHERE id = 1 AND balance >= 100;
-- If this affects 0 rows, the sender lacked the funds. Roll back.

UPDATE accounts
SET balance = balance + 100
WHERE id = 2;

COMMIT;
```

Two details matter most:

- `balance = balance - 100` does the math inside the engine. There is no gap between the read and the write for another transaction to get in between, so the lost update from section 6 cannot happen.

- `AND balance >= 100` puts the funds check in the same statement as the write. Checking with a separate `SELECT` first leaves a window where another transaction can empty the account between the check and the update.

The pattern applies well past money. Any time we are tempted to write "read the row, decide in our own code, write it back", ask whether the decision can move into the `WHERE` clause instead.

10. The same transfer in Java

Plain JDBC first, because it shows every step the database actually needs:

```
public void transfer(long from, long to, long cents) throws SQLException {
    try (Connection conn = dataSource.getConnection()) {
        conn.setAutoCommit(false);          // this is the BEGIN
        try {
            int rows;
            try (PreparedStatement ps = conn.prepareStatement("""
                    UPDATE accounts SET balance_cents = balance_cents - ?
                    WHERE id = ? AND balance_cents >= ?
                    """)) {
                ps.setLong(1, cents);
                ps.setLong(2, from);
                ps.setLong(3, cents);
                rows = ps.executeUpdate();
            }

            if (rows == 0) {
                throw new InsufficientFundsException(from);
            }

            try (PreparedStatement ps = conn.prepareStatement("""
                    UPDATE accounts SET balance_cents = balance_cents + ?
                    WHERE id = ?
                    """)) {
                ps.setLong(1, cents);
                ps.setLong(2, to);
                ps.executeUpdate();
            }

            conn.commit();
        } catch (Exception e) {
            conn.rollback();
            throw e;
        }
    }
}
```

Production code normally lets Spring manage the boundary instead:

```java
@Service
public class TransferService {

    private final JdbcTemplate jdbc;

    public TransferService(JdbcTemplate jdbc) {
        this.jdbc = jdbc;
    }

    @Transactional(isolation = Isolation.READ_COMMITTED)
    public void transfer(long from, long to, long cents) {
        int rows = jdbc.update("""
                UPDATE accounts SET balance_cents = balance_cents - ?
                WHERE id = ? AND balance_cents >= ?
                """, cents, from, cents);

        if (rows == 0) {
            // Unchecked, so Spring rolls the transaction back.
            throw new InsufficientFundsException(from);
        }

        jdbc.update("""
                UPDATE accounts SET balance_cents = balance_cents + ?
                WHERE id = ?
                """, cents, to);
    }
}
```

Points an interviewer may ask about:

- `setAutoCommit(false)` is what opens the transaction in JDBC. Without it every statement commits on its own, and the two updates are no longer one unit.
- **Spring rolls back on unchecked exceptions only.** `RuntimeException` and `Error` trigger a rollback; a checked exception does not, so the partial work commits. `InsufficientFundsException` therefore extends `RuntimeException`, and a checked one would need `@Transactional(rollbackFor = ...)`. This is the most common transaction bug in Spring code.
- **`@Transactional` works through a proxy**, so calling `transfer` from another method of the same class skips the proxy and starts no transaction at all. Self-invocation silently losing the transaction is the second most common bug, and it comes up often in interviews.
- Every statement must run on the same connection. `JdbcTemplate` takes it from the transaction context, but a call that opens its own connection from the `DataSource` runs outside the transaction and will not roll back with it.
- Money is `long` cents, or `BigDecimal`, and never `double`. Floats lose small amounts on every operation and those errors grow.
- This runs at read committed, which is enough here because the check sits in the `WHERE` clause. Raise it to `Isolation.SERIALIZABLE` and the commit itself can fail with SQLSTATE `40001`, which Spring

surfaces as `CannotSerializeTransactionException`. That is a normal error to retry rather than a bug, and the retry has to run the whole method again. Because the annotation is proxy-based, the retry has to come from outside the bean, such as `@Retryable` on the caller.

11. ACID across services

ACID is one database's promise about one connection. It does not stretch across two services that own separate databases, and no `BEGIN` spans them. Three answers are common.

Two-phase commit. A coordinator asks every participant to prepare, then tells them all to commit. It works, but it is rare in practice: participants hold locks while they wait for the coordinator, and a coordinator that dies between the two steps leaves them holding those locks with no instruction.

Saga. Break the work into a sequence of local transactions, each paired with a compensating transaction that undoes it. Reserve the room, then charge the card; if the charge fails, the compensating step releases the room. Nothing isolates the steps from each other, so other readers see the half-finished states. Recovery is ours to design.

Transactional outbox. The one to use first, because it solves the common case cleanly. "Update the database and publish an event" spans two systems, which is why it cannot be atomic. The fix is to make it one system: write the row and an outbox row in the *same* local transaction, then let a separate relay read the outbox and publish to the broker.

```
BEGIN;
  UPDATE orders SET status = 'shipped' WHERE id = 12345;
  INSERT INTO outbox (topic, payload)
  VALUES ('order.shipped', '{"order_id": 12345}');
COMMIT;
-- A relay process publishes the outbox rows, then marks them sent.
```

The database's own atomicity now makes sure the state change and the plan to publish are saved together. Without it, a crash between the commit and the publish call loses the event permanently. Publishing first has the opposite problem: it announces something that may then fail to commit. The relay can also crash after publishing but before marking the row sent, so the same event can arrive twice and consumers have to be idempotent, meaning safe to apply more than once.

12. ACID and BASE

BASE is the label for the opposite trade-off: **B**asically **A**vailable, **S**oft state, **E**ventually consistent. Reads may return stale data for a while, and the system settles once writes stop. Plenty of distributed stores and caches belong here, and the choice is about what the data is worth rather than which acronym is better. Money and inventory need ACID. A view counter does not.

One trap to watch for:

> **The two C words are unrelated**
>
> Consistency in ACID means the constraints we wrote down still hold after a transaction. Consistency in the CAP theorem means every read sees the most recent write, which distributed systems people call linearizability. Same word, different property. A system can have one and not the other, and an interviewer who asks "is your system consistent?" may be asking about either.

13. Interview questions

Each answer below is written to be said out loud as it stands. The last sentence always leads toward the next question rather than closing the topic.

"What does ACID stand for?"

> ACID is four promises a database makes about a transaction.
>
> - **Atomicity.** Every statement in the transaction completes, or none of them do.
> - **Consistency.** The database moves from one valid state to another, so all the constraints still hold afterwards.
> - **Isolation.** Transactions running at the same time do not see each other's unfinished work.
> - **Durability.** Once a commit returns, the change survives a crash.
>
> The one worth talking about is consistency, because it is really the result of the other three plus the constraints we declared. Isolation is the one we can adjust, and that is usually where the follow-up questions lead.

"Which letter is different from the other three?"

> Consistency. The other three are things the engine does for us, and consistency is partly ours. The database enforces the constraints we declared, such as foreign keys and check constraints, but it cannot enforce a rule we never wrote down. If "every order has at least one line item" lives only in our service code, nothing in the database stops a transaction from breaking it. So consistency is the outcome, and atomicity, isolation and durability are how we get there.

"What is a dirty read?"

> A dirty read is when one transaction reads a row that another transaction has changed but not committed yet. If that second transaction rolls back, we have acted on a value that never existed. It is the anomaly that read uncommitted allows and read committed blocks. Worth adding: Postgres never produces a dirty read at any level. Asking for read uncommitted there is accepted, but the transaction still runs as read committed.

"What is the default isolation level?"

> It depends on the database, so that is worth checking first. Postgres defaults to read committed, and MySQL with InnoDB defaults to repeatable read. The difference matters when we move code between them, because the same queries can behave differently with nothing in them changed. Postgres repeatable read is also snapshot isolation, so it blocks phantom reads as well, which is stricter than the standard asks for.

"How would you prevent two users double-spending the same balance?"

> The rule is to never read the balance into application code, decide there, and write it back, because that gap between the read and the write is where the second user gets in. Instead we do both in one statement: update the row, set balance to balance minus one hundred, where the id matches and the balance is at least one hundred. The subtraction happens inside the engine and the funds check is part of the same statement, so the row lock forces the two transactions to run one after the other. Then we check the affected row count, and zero rows means there were not enough funds, so we roll the transaction back. If a decision really cannot be written as a condition, we take an explicit lock with `SELECT ... FOR UPDATE`, and optimistic locking with a version column is the other option when clashes are rare.

"How do you keep a database write and a published event in sync?"

> They are two separate systems, so no transaction covers both. If we commit and then the publish call fails, the event is lost, and if we publish first and then the commit fails, we have announced something that never happened. The outbox pattern turns it into one system: in the same local transaction we write the row and also insert a row into an outbox table, then a separate relay process reads that table, publishes to the broker, and marks the row as sent. The database's own atomicity makes sure both rows are saved together. The relay can still crash after publishing but before marking the row sent, so the same event can arrive twice, which means consumers have to be idempotent.

"Why not run everything at serializable?"

> It costs us concurrency, and it adds work in the application. Postgres uses serializable snapshot isolation, which tracks which live transactions read and wrote the same data, then cancels one of them when they conflict. The result is a serialization failure at commit time, SQLSTATE `40001`. That is a normal outcome rather than a bug, but every caller now needs retry logic, and the retry has to run the whole transaction again rather than just the failed statement. So serializable is right where correctness under concurrent writes matters more than speed, and read committed is usually enough elsewhere, especially when the check already sits in the `WHERE` clause.

"What happens if the server loses power mid-transaction?"

> Anything uncommitted rolls back during recovery, and anything committed is replayed from the write-ahead log. The engine writes a note about every change to that log and calls `fsync` before the commit returns, so the record is on disk even if the data pages were still in memory. That is what makes durability real. Worth adding: the promise depends on that `fsync` actually happening, and both engines have a setting that relaxes it for speed. With those switched on we can lose the last fraction of a second of committed work in a crash.

14. Common mistakes

- Long-running transactions hold their snapshot open. In Postgres that stops vacuum from clearing out dead rows, and the table fills up with them. Open the transaction late, commit early, and never leave one open across a call to an external API.
- `SELECT ... FOR UPDATE` holds its row locks until the transaction commits, not just for the length of the query. Two code paths that lock the same rows in a different order will deadlock. Pick one lock order and use it everywhere.
- The database detects deadlocks and kills one of the transactions. That is normal, not a fault, and the caller retries. Treating it as fatal produces pager alerts nobody needs to act on.
- Retry logic must run the whole transaction again from the top. In Postgres, retrying only the failed statement returns `current transaction is aborted` for every statement after it, until we roll back and start again.
- ORMs open and close transactions for us. Knowing where those boundaries actually fall matters more than knowing the ORM's API, especially when a lazily loaded field fires a query after the transaction has already closed.
- Connection pools hand out reused connections. An isolation level or session setting changed on one connection leaks into whatever runs on it next.

15. Quick reference

Term	One line
Atomicity	Every statement completes, or none of them do
Consistency	Constraints hold before and after
Isolation	Transactions running at once do not see each other's unfinished work
Durability	Committed means it survives a crash
Dirty read	Reading uncommitted data that may be rolled back
Non-repeatable read	Same row read twice, different values
Phantom read	Same query run twice, different set of rows
Lost update	Two transactions read then write, one write overwrites the other

Term	One line
Write skew	Two transactions each check a rule, both commit, the rule breaks
MVCC	Keep row versions instead of locking readers out
WAL	Append-only log flushed to disk before commit returns
Snapshot isolation	Transaction reads one fixed view of the database
SSI	Postgres serializable, cancels clashing transactions with `40001`
Saga	Sequence of local transactions with undo steps
Outbox	Write state and event in one local transaction, relay publishes later
BASE	Basically available, soft state, eventually consistent

Task	Syntax
Start a transaction	`BEGIN;`
Commit	`COMMIT;`
Undo	`ROLLBACK;`
Partial undo	`SAVEPOINT s; ... ROLLBACK TO SAVEPOINT s;`
Set level	`BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;`
Session default	`SET SESSION CHARACTERISTICS AS TRANSACTION ISOLATION LEVEL REPEATABLE READ;`
Lock rows for update	`SELECT * FROM t WHERE id = 1 FOR UPDATE;`
Safe decrement	`UPDATE t SET n = n - 1 WHERE id = 1 AND n >= 1;`
Check current level (Postgres)	`SHOW transaction_isolation;`
Check current level (MySQL)	`SELECT @@transaction_isolation;`